

Parallel Agent Coordination — Deep-Web Research (SP5)

Per §11.4.99 this document cites the latest authoritative online sources (URLs + access date in the ## Sources verified footer). Claims are grounded; honest negatives are flagged inline.

- [1. What exists in-repo today \(ground truth\)](#)
- [2. Multi-agent coordination frameworks](#)
- [3. Durable / parallel task orchestration \(Temporal, Ray, Dask\)](#)
- [4. Go worker-pool / actor patterns + the concurrency submodule](#)
- [5. The convergent “operation = parallel-dispatchable unit” abstraction](#)
- [6. Isolation for parallel mutation \(worktree / CoW / scoped locks\)](#)
- [7. Recommendation — single shared substrate for HelixCode + HelixAgent](#)
- [8. Summary](#)
- Sources verified 2026-06-10

Three in-repo artefacts bound the build-vs-reuse decision (read read-only this session):

Artefact	Path
concurrency submodule (digital.vasic.concurrency, Go 1.24+)	submodules/concurrency/ (pkg/ {pool,queue,limiter,breaker,semaphore,monitor}

Artefact	Path
agentic submodule (digital.vasic.agentic, Go 1.24+, ~2.6k LOC)	submodules/agentic/
coordinator.go (in-app, duplicated)	helix_code/internal/agent/coordinator.go

Finding (FACT): concurrency already provides the worker-pool/priority-queue/semaphore/breaker primitives the literature converges on; agentic already provides the DAG + state + checkpoint layer. coordinator.go re-implements a thin slice of both with no parallel drain and its own capability-matching — exactly the “duplicated coordinator.go vs agentic stacks” the SP5 brief flags.

2. Multi-agent coordination frameworks

How the leading agentic stacks decompose work into parallel units and coordinate them. All are Python (the agentic ecosystem is Python-first); HelixCode is Go, so these inform the *interface design*, not a drop-in dependency.

Framework	Repo / license / lang	Decomposition unit	Coordination primitive
LangGraph	github.com/langchain-ai/langgraph · MIT · Python (JS port exists)	Node in a StateGraph (each node a function; subgraphs are full graphs compiled as nodes)	Centralized shared state threaded through nodes; fan-out = one node triggers multiple downstream nodes; scatter-gather with a barrier (downstream waits until all parallel branches complete); supervisor pattern via nested subgraphs. The dominant 2026 default for stateful workflows.
CrewAI	github.com/crewAIInc/crewAI · MIT · Python	Role-playing Agent (role/goal/backstory) + Task, assembled into a Crew	Supervisor (subagents one level deep), fan-out (parallel subagent dispatch), pipeline (sequential tasks). YAML-driven role delegation.
AutoGen	github.com/microsoft/autogen · MIT (CC-BY for docs) · Python (.NET port)	Conversable agent in a multi-agent conversation	Message-passing conversation patterns (group chat, nested chat); a manager/orchestrator selects the next speaker. Strong for offline quality-sensitive flows.
OpenAI Agents SDK (replaced Swarm, GA Mar 2025)	github.com/openai/openai-agents-python · MIT · Python	Agent (instructions + model + tools + allowed-handoffs)	Named primitives: Agent / Runner / Handoffs / Tools / Sessions / Guardrails . Runner is the built-in agent loop (tool-call → result → repeat until

Framework	Repo / license / lang	Decomposition unit	Coordination primitive
			done); Handoff = explicit agent-to-agent control transfer carrying context; Sessions = persistent memory; Guardrails run <i>in parallel</i> with execution. Subagents (parent spawns specialized children) formalized 2026. Closest to a minimal, named substrate.
OpenAI Swarm	github.com/openai/swarm · MIT · Python	Lightweight agent + handoff	Educational/ superseded by Agents SDK; the handoff abstraction is the lasting contribution.
MetaGPT	github.com/FoundationAgents/ MetaGPT (was geekan/ MetaGPT) · MIT · Python	Role agent governed by an SOP (predefined workflow)	Shared message pool + publish/ subscribe : agents publish structured messages, subscribe by profile — decouples producers from consumers (no constant peer-to-peer). The pub/sub message pool is the reusable coordination idea for HelixCode’s event bus.

Convergent patterns (the “5 that work in 2026”): **fan-out** (parallel dispatch + gather), **pipeline** (sequential stages), **supervisor** (a coordinator routes/delegates, one level deep), **debate** (multiple agents argue to consensus), **swarm** (peer handoffs, no central router). HelixCode’s own `debate_orchestrator` submodule already maps to “debate”; `agent ic`’s DAG covers “pipeline” + “fan-out”; `coordinator . go`’s registry maps to “supervisor”.

3. Durable / parallel task orchestration (Temporal, Ray, Dask)

How a unit of work becomes *independently dispatchable + retried + isolated* — the durability/scheduler layer below the agent layer.

- **Temporal** (github.com/temporalio/temporal + go.temporal.io/sdk, MIT, Go SDK first-class). Unit = an **Activity** (a function that the SDK retries automatically per a configurable **RetryPolicy**, with timeouts) driven from a **Workflow** (durable, replayable, fault-tolerant — full running state survives worker crash). Dispatch = **Task Queues** (lightweight, dynamically allocated; Workflow/Activity/Nexus task queues that Workers poll). When a worker dies the tasks persist in the queue until a worker recovers. **This is the canonical “operation = durable retried dispatchable unit” model and it has a native Go SDK** — directly relevant to a Go HelixCode substrate.
- **Ray** (github.com/ray-project/ray, Apache-2.0, Python/C++ core). Two units: **Task** (stateless remote function) and **Actor** (stateful remote object — keeps state collocated with compute). **Fully distributed, decoupled control plane + scheduler** (no single master bottleneck) — scales to millions of tasks/s. The Task-vs-Actor split is the key design lesson: stateless agent operations → tasks, stateful long-lived agents → actors.
- **Dask** (github.com/dask/dask, BSD-3, Python). Unit = a node in a **lazily-built task graph** optimized before execution; a **centralized scheduler** coordinates (throughput ceiling ~3k tasks/s on 512 cores — the centralized-scheduler bottleneck Ray was designed to escape). Best for data-parallel pipelines, weaker for fine-grained agent dispatch.

Lesson for HelixCode: Temporal’s Activity+TaskQueue+RetryPolicy and Ray’s Task-vs-Actor split are the durability/scheduling vocabulary. A decentralized scheduler (Ray) beats a centralized one (Dask) for fine-grained dispatch; durable task-queue persistence (Temporal) is the recovery story. HelixCode does NOT need to adopt Temporal/Ray as dependencies — it needs to mirror their *interface shape* on top of the existing concurrency worker pool.

4. Go worker-pool / actor patterns + the concurrency submodule

The submodules/concurrency README/docs (read this session) confirm it already implements the Go-idiomatic versions of these patterns:

- **Worker pool:** `pool.NewWorkerPool(&pool.PoolConfig{Workers, QueueSize, TaskTimeout, ShutdownGrace, OnError})` with `Submit / SubmitWait(ctx) / SubmitBatch(channel of results) / ParallelExecute / generic Map[T,R](ctx, items, parallelism, fn)`. `pool.Task` interface = `ID() string + Execute(ctx) (interface{}, error)` — **this is already the minimal dispatchable-unit interface.**
- **Priority queue:** `queue.New[T](capacity) generic, 4 levels (Critical/High/Normal/Low)`, heap with stable FIFO tie-break — directly supports §11.4.42/§11.4.72 priority-first + audio-first dispatch ordering.
- **Weighted semaphore** (`semaphore.New(weight) + Acquire(ctx, n) / Release(n)`) — the resource-budget primitive for §11.4.119 single-resource-owner partitioning and §12.6 memory-gated concurrency.
- **Circuit breaker + token-bucket/sliding-window limiters + resource monitor** (CPU/mem/disk via `gopsutil`) — resilience + back-pressure, mirroring Temporal’s retry/timeout story.

The classic Go actor pattern (a goroutine owning private state, reachable only via a channel) is the natural fit for Ray-style stateful “actor” agents on top of this — but concurrency does NOT ship an actor/mailbox primitive today (honest gap). Per §11.4.74 extend-don’t-reimplement, an `pkg/actor` (goroutine + typed mailbox + supervisor) belongs **inside** concurrency, not in a consuming project.

5. The convergent “operation = parallel-dispatchable unit” abstraction

Synthesizing §2–§4, the literature converges on a four-role interface. Naming aligned to what already exists in-repo so `HelixCode` + `HelixAgent` consume ONE substrate:

Role	Convergent meaning (Temporal / Ray / LangGraph / Agents SDK)	In-repo anchor to build on
Unit	A self-describing piece of work: id, payload, declared capability/resource requirements, priority, idempotency key. (Temporal Activity, Ray Task, LangGraph node, Agents-SDK tool/handoff target.)	Extend concurrency <code>pool.Task(ID() + Execute(ctx))</code> with <code>Requires()</code> <code>Capabilities + Priority() + ResourceClass()</code> .
Scheduler	Decides WHICH units run now + in what order, honoring priority, resource budget, and a decentralized (not single-master) control plane.	concurrency <code>queue.PriorityQueue[Unit]</code> + semaphore for budget + monitor for §12.6 memory gating.
Dispatcher	Maps a ready unit onto an executor (worker goroutine, subagent, or remote worker) with retry/timeout/circuit-breaking + isolation.	concurrency <code>pool.WorkerPool</code> + breaker + limiter; isolation layer per §6.
Resolver	Picks the right executor for a unit (capability match) and merges/reconciles results from parallel units (conflict resolution, voting, scatter-gather barrier).	This is the genuinely NEW layer. <code>coordinator.go</code> ’s <code>findSuitableAgent + ResolutionMethodVoting</code> is a first draft; the agentic <code>WorkflowState</code> + barrier is the merge half.

The single substrate = `Unit` + `Scheduler` + `Dispatcher` + `Resolver` layered as:
concurrency (Scheduler+Dispatcher mechanics) ← `agentic` (DAG/state/checkpoint = the orchestration shape) ← a thin new **coordination capability** (`Unit` + `Resolver`: capability-match + result-merge + isolation policy) that BOTH `helix_code/internal/agent/coordinator.go` AND `HelixAgent` import instead of each re-implementing. Per §11.4.74/

§11.4.51 it is a decoupled, project-not-aware own-org submodule consumed from the project root — NOT a fourth in-app coordinator.

6. Isolation for parallel mutation (worktree / CoW / scoped locks)

Maps directly to §11.4.58 (PWU disjoint scope) / §11.4.84 (quiescence) / §11.4.119 (single-resource-owner). 2025–2026 best practice for parallel AI agents mutating a shared tree:

1. **One task → one branch → one git worktree → one agent** (the canonical invariant). `git worktree add` (Git ≥ 2.5) gives each agent an isolated working directory sharing ONE `.git` object store — creation is instant, disk is near-free (CoW-like via shared objects). Changes in one worktree are invisible to others; conflicts surface as *standard git conflicts* instead of invisible runtime corruption + `.git/index.lock` races. Native worktree support landed in Claude Code / Cursor / Copilot CLI (Oct 2025–Feb 2026). This is exactly the concurrency/agent-ic “git worktree add per subagent” recommendation already in §11.4.84.
2. **Decompose by domain/feature boundary**, never split work that touches the same files from different directions (the §11.4.58 disjoint-PWU rule).
3. **Runtime isolation above the worktree** when >4 concurrent agents: one lightweight container per worktree (isolated DB/dev-server) — emerging production standard. For HelixCode this is the §11.4.76 `containers` submodule per agent.
4. **Scoped locks for genuinely-shared exclusive resources**: a worktree isolates the *filesystem* but NOT a shared DB row, a single HDMI/audio sink, or a device handle — those need the §11.4.119 single-owner advisory lock (claim-when-free, release-immediately) + concurrency semaphore. The `.git/MUTATION_IN_PROGRESS` lockfile (§11.4.84) is the same pattern for mutation gates.
5. `git sparse-checkout` per worktree to constrain each agent’s tree to the files it needs (monorepo I/O containment).

Practical cap: 8–10 concurrent worktrees before management overhead exceeds parallelism benefit — consistent with the §11.4.58 6-agent / §12.6 60%-memory caps already in governance.

7. Recommendation — single shared substrate for HelixCode + HelixAgent

REUSE, then EXTEND — do not build a fourth coordinator (per §11.4.74 catalogue-first):

1. **Keep concurrency as the Scheduler+Dispatcher floor** (worker pool + priority queue + semaphore + breaker + limiter + monitor). It already matches the Temporal/Ray mechanical model. **Extend it** (PR upstream, §11.4.74) with: (a) a richer `Unit` interface (`Requires()/Priority()/ResourceClass()` over the existing `pool.Task()`); (b) a new `pkg/actor` (goroutine + typed mailbox + supervisor) for Ray-style stateful agents — the honest gap from §4.
2. **Keep agent-ic as the orchestration shape** (DAG + `WorkflowState` + checkpoints + `NodeTypeParallel` fan-out + scatter-gather barrier). **Extend it** so `NodeTypeParallel` drains onto a concurrency worker pool with a real scheduler (today the link is thin), and so checkpoints copy messages+history not just variables (§4 gap).
3. **Build the genuinely-new thin Resolver/Unit layer once** — capability-match + result-merge (voting/scatter-gather) + isolation policy — as a decoupled own-org submodule (or a `pkg/` inside `agent-ic`), and have **both** `helix_code/internal/agent/`

`coordinator.go` AND `HelixAgent` consume it. This **retires the duplication**: `coordinator.go`'s `findSuitableAgent + ResolutionMethodVoting + circuit-breaker/retry` get re-expressed as consumers of the shared substrate, not re-implementations.

4. **Adopt named primitives from the Agents SDK + MetaGPT** for the agentic surface: **Handoff** (explicit unit-to-unit control transfer carrying context), **Guardrails** (validation in parallel with execution), and a **shared pub/sub message pool** (MetaGPT) — `HelixCode` already has an `event_bus` submodule to host the pub/sub coordination.
5. **Isolation = git worktree per subagent + containers per worktree above 4 agents + semaphore/single-owner lock for shared exclusive resources** (§6) — this is already the §11.4.58/§11.4.84/§11.4.119 governance; the substrate just needs to enforce it mechanically (claim a worktree + a resource token before dispatch).

Net: `HelixCode`'s parallel-agent story is **~80% already built** across concurrency + agentic; the missing 20% is (a) wiring agentic's parallel node to concurrency's scheduler, (b) one shared `Unit+Resolver` layer replacing the in-app `coordinator.go` duplication, (c) a `pkg/actor` for stateful agents, (d) mechanical worktree+resource-token isolation.

8. Summary

1. SP5 substrate = **Unit + Scheduler + Dispatcher + Resolver** — the four roles every surveyed stack (Temporal, Ray, LangGraph, OpenAI Agents SDK, MetaGPT) converges on.
2. **Unit** = self-describing work (id, capability/resource reqs, priority, idempotency) — extend concurrency's existing `pool.Task(ID()+Execute(ctx))`.
3. **Scheduler** = `concurrency.queue.PriorityQueue[Unit] + semaphore budget + monitor` (§12.6 memory gate); decentralized beats Dask's centralized scheduler.
4. **Dispatcher** = `concurrency.pool.WorkerPool + breaker + limiter + retry` — mirrors Temporal Activity/TaskQueue/RetryPolicy (native Go SDK exists, so the shape is proven in Go).
5. **Resolver** = the genuinely-new layer: capability-match + scatter-gather/voting result-merge + isolation policy.
6. **REUSE not rebuild**: concurrency already ships worker-pool/priority-queue/semaphore/breaker; agentic already ships the DAG+state+checkpoint+parallel-node layer. ~80% exists.
7. **Retire the duplication**: `helix_code/internal/agent/coordinator.go` re-implements capability-match + resilience + a never-drained `taskQueue`; re-express it as a CONSUMER of the shared substrate.
8. **EXTEND (§11.4.74)**: wire agentic `NodeTypeParallel` to a real concurrency scheduler; add `pkg/actor` (goroutine+mailbox) for Ray-style stateful agents; checkpoint messages+history not just variables.
9. **Adopt named primitives**: Handoff + Guardrails (Agents SDK) + shared pub/sub message pool (MetaGPT, host on the `event_bus` submodule).
10. **Isolation strategy**: one task→one branch→one **git worktree**→one agent (CoW-like, instant, conflicts surface as git conflicts) — §11.4.58/§11.4.84.
11. **Above 4 agents**: one `containers`-submodule container per worktree (§11.4.76); sparse-checkout to bound each tree; cap 6–10 agents (§11.4.58 / §12.6).
12. **Shared exclusive resources** (DB row, single sink, device handle) need a §11.4.119 single-owner advisory lock + concurrency semaphore — worktree isolation alone does NOT cover them.

Sources verified 2026-06-10

- LangGraph vs AutoGen vs CrewAI architecture comparison — <https://latenode.com/blog/platform-comparisons-alternatives/automation-platform-comparisons/langgraph-vs-autogen-vs-crewai-complete-ai-agent-framework-comparison-architecture-analysis-2025> (accessed 2026-06-10)
- LangGraph multi-agent orchestration + supervisor/fan-out/shared-state — <https://latenode.com/blog/ai-frameworks-technical-infrastructure/langgraph-multi-agent-orchestration/langgraph-multi-agent-orchestration-complete-framework-guide-architecture-analysis-2025> (accessed 2026-06-10)
- Swarm vs Supervisor multi-agent architecture — <https://www.augmentcode.com/guides/swarm-vs-supervisor> (accessed 2026-06-10)
- Multi-agent orchestration: 5 patterns (fan-out/pipeline/debate/supervisor/swarm) — <https://www.digitalapplied.com/blog/multi-agent-orchestration-5-patterns-that-work> (accessed 2026-06-10)
- OpenAI Agents SDK docs (Agent/Runner/Handoffs/Tools/Sessions/Guardrails) — <https://openai.github.io/openai-agents-python/> (accessed 2026-06-10)
- OpenAI Swarm (educational, superseded) — <https://github.com/openai/swarm> (accessed 2026-06-10)
- MetaGPT shared message pool + pub/sub + SOP/roles — <https://arxiv.org/html/2308.00352v6> (accessed 2026-06-10)
- Temporal durable execution / Activity / Task Queue / RetryPolicy — <https://docs.temporal.io/task-queue> and <https://docs.temporal.io/activity-execution> and <https://pkg.go.dev/go.temporal.io/sdk/temporal> (accessed 2026-06-10)
- Ray actor/task model, decentralized scheduler — <https://arxiv.org/pdf/1712.05889> and <https://docs.ray.io/en/latest/ray-more-libs/dask-on-ray.html> (accessed 2026-06-10)
- Ray vs Dask (centralized vs decentralized scheduler, throughput) — <https://www.oreateai.com/blog/dask-vs-ray-choosing-the-right-framework-for-distributed-computing/1c4610b09094611aca8b1969c1beb83e> (accessed 2026-06-10)
- Git worktree isolation for parallel AI agents (one task→one branch→one worktree; runtime isolation; caps) — <https://www.augmentcode.com/guides/git-worktrees-parallel-ai-agent-execution> and <https://www.mindstudio.ai/blog/git-worktrees-parallel-ai-coding-agents> (accessed 2026-06-10)

Honest negatives: (a) the agentic-framework comparison sources are vendor/secondary blogs, not the primary framework docs for every claim — primary repos cited where load-bearing (Swarm, MetaGPT arXiv, Temporal/Ray docs). (b) No source claims a Go-native equivalent of LangGraph/CrewAI exists at production grade — the Go agentic substrate for HelixCode is genuinely original work built on the `concurrency+agentic` submodules, not a reusable off-the-shelf Go framework. (c) `concurrency` ships no actor/mailbox primitive today — the `pkg/actor` recommendation is a gap-fill, not a description of existing code.